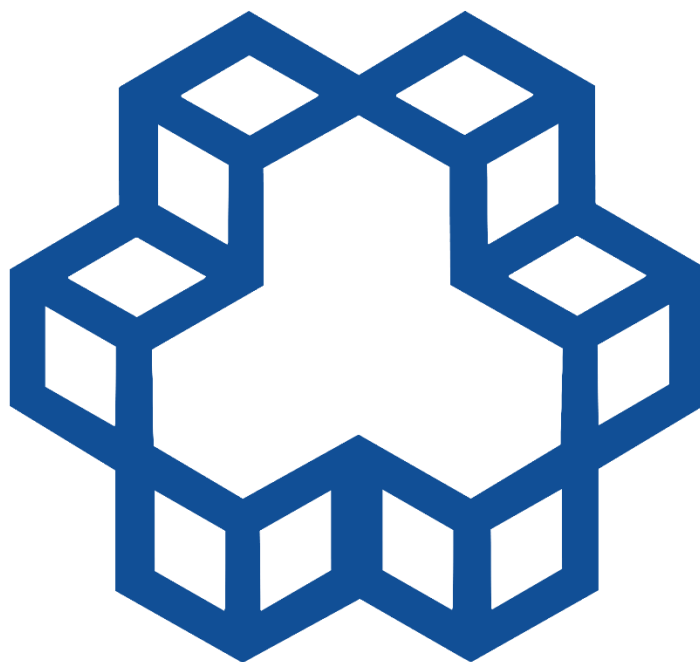


شبکه های کامپیوتری 1

بهار 1405



دانشجو:

علی کاشی پزها

شماره دانشجویی:

40224641

استاد درس:

دکتر مرادیان

پروژه درسی

۱. مقدمه و دامنه کار

هدف این پروژه پیاده‌سازی یک مسیر ارتباطی کامل است: داده خام در بالاترین لایه رمزنگاری می‌شود، در لایه پیوند داده فریم‌بندی و در برابر خطا مقاوم می‌گردد، از یک کانال فیزیکی نویزی می‌گذرد و در گیرنده تمام این مراحل معکوس می‌شوند تا داده سالم بازیابی شود.

معیار طراحی در سراسر پروژه این بوده که هیچ سازوکاری «تشریفاتی» نباشد. هر مکانیزمی که پیاده شده باید در شرایطی که برای آن ساخته شده است واقعاً به کار بیفتد و اثرش قابل مشاهده و اندازه‌گیری باشد. به همین دلیل رابط گرافیکی تنها یک نمایش تزئینی نیست؛ هر سنج در آن با شماره بند صورت مسئله برچسب خورده است تا هر نیازمندی به کد و به شاهد عینی آن قابل ردیابی باشد.

۲. معماری برنامه

معماری به‌صورت لایه‌ای و با مرزهای مشخص پیاده شده است. هر لایه فقط از طریق داده به لایه پایین‌تر وابسته است و هیچ لایه‌ای به جزئیات درونی لایه دیگر دسترسی ندارد. این جداسازی باعث شد بتوان مثلاً مدل خطای کانال سیمی را کاملاً عوض کرد بدون آنکه یک خط از لایه پیوند داده تغییر کند.

مسیر ارسال: متن ورودی ← رمزنگاری چرخشی (XOR) (security.py) ← قطع‌بندی به بار مفید فریم ← سرآیند-CRC + 16 سرآیند CRC-32 + بدنه ← کدگذاری ← Hamming (12,8) ← Bit Stuffing افزودن پرچم‌ها ← کدینگ خطی یا مدولاسیون ← کانال

مسیر دریافت: کانال ← آشکارسازی/دیکد خط ← یافتن مرز فریم با جست‌وجوی پرچم ← Bit Unstuffing ← تصحیح Hamming بررسی CRC-16 سرآیند ← بررسی ← CRC-32 بافر پنجره دریافت و تحویل مرتب ← رمزگشایی ← متن اصلی

۲.۱. ماژول‌ها

بند	مسئولیت	فایل
5.3	رمز XOR چرخشی با کلید پویا	security.py
4.1, 4.2	فریم‌بندی، Bit Stuffing، مرزبندی فریم، CRC-32، CRC-16 و Hamming (12,8)	data_link_layer.py
2, 3	AWGN، مدل نویز، QAM16، BPSK، HDB3، B8ZS	physical_layer.py
4.3	Selective Repeat، پنجره لغزان، تایمر هر فریم، ACK/NAK	arq_protocol.py
5.1	جاروب Eb/N0 و اندازه پنجره، تولید نمودارها	performance_analysis.py
5.2	داشبورد سه‌تایی زمان واقعی	gui.py

۲.۲. چرا شبیه‌سازی گسسته و تک‌محور است

هسته پروتکل بر پایه یک ساعت گسسته («تیک») کار می‌کند. در هر تیک، فریم‌های در حال گذر جابه‌جا می‌شوند، تایمرها یک واحد کم می‌شوند و رخدادهای رسیده پردازش می‌شوند. دلیل این انتخاب آن است که تأخیر انتشار، مهلت تایم‌اوت و اندازه پنجره سه کمیتی هستند که رفتار Selective Repeat از رابطه میان آن‌ها بیرون می‌آید؛ با یک ساعت گسسته این رابطه صریح و تکرارپذیر می‌شود، در حالی که با زمان‌بندی بر پایه ساعت دیواری نتایج به سرعت اجرای ماشین وابسته می‌شد و مقایسه نمودارها بی‌معنا.

یک نکته مهم درباره تکرارپذیری: تولید نویز از یک Generator بلندعمر انجام می‌شود که یک بار ساخته و در تمام ارسال‌ها به اشتراک گذاشته می‌شود. اگر به جای آن در هر ارسال با یک seed ثابت مولد تازه ساخته شود، هر بازارسال دقیقاً همان نویز قبلی را می‌بیند و فریمی که یک بار خراب شده هرگز درست نمی‌رسد — یعنی ARQ در ظاهر کار می‌کند ولی هیچ‌گاه موفق نمی‌شود. استقلال آماری نویز میان بازارسال‌ها شرط لازم درستی این شبیه‌سازی است.

۳. پیاده‌سازی کانال‌ها

۳.۱. کانال سیمی: کدینگ خطی B8ZS و HDB3

در کانال سیمی داده به سه سطح -1, 0, +1 نگاشت می‌شود (سیگنالینگ دوقطبی). AMI مشکل اصلی ساده این است که رشته طولانی صفر هیچ پالسی روی خط تولید نمی‌کند و گیرنده همزمانی ساعت خود را از دست می‌دهد. هر دو کد این مشکل را با «نقض عامدانه» قاعده قطبیت حل می‌کنند:

- **B8ZS:** هر هشت صفر متوالی با الگوی 000VB0VB جانشین می‌شود، که در آن V پالس ناقض قطبیت و B پالس سازگار است. نتیجه این است که طول رشته بدون پالس هرگز از 7 بیشتر نمی‌شود.
- **HDB3:** با جانشینی هر چهار صفر، همان تضمین را سخت‌تر می‌کند و سقف رشته بدون پالس را به 3 می‌رساند؛ در عوض به حافظه بیشتری نیاز دارد چون باید تعداد پالس‌ها از آخرین جانشینی را بشمارد تا توازن DC حفظ شود.

این دو تضمین در رابط گرافیکی مستقیماً اندازه‌گیری و نمایش داده می‌شوند: خانه «Line code health» هم توازن DC جمع‌ی و هم بلندترین رشته بدون پالس را گزارش می‌کند و در اجرای واقعی مقدار آن برای B8ZS دقیقاً ۷ و برای HDB3 دقیقاً ۳ مشاهده می‌شود. بنابراین ادعای تئوری در همان لحظه اجرا راستی‌آزمایی می‌شود.

اصلاح مدل خطای کانال سیمی: مدل اولیه خطا را با «قرینه کردن نماد» اعمال می‌کرد. اما قرینه صفر خود صفر است، پس هر نماد صفر عملاً از نویز مصون می‌ماند — و در یک خط دوقطبی بیشتر نمادها صفرند. نتیجه آن بود که کانال سیمی به‌طور مصنوعی بسیار تمیزتر از واقعیت رفتار می‌کرد. مدل فعلی هر نماد آسیب‌دیده را به یکی از دو سطح دیگر منتقل می‌کند، پس هم «گم شدن پالس» و هم «پالس جعلی» ممکن است. نرخ خطا نیز از رابط گرافیکی قابل تنظیم شد تا این رفتار قابل مشاهده باشد.

۳.۲. کانال بی‌سیم: QAM16، BPSK و نویز AWGN

در کانال بی‌سیم بیت‌ها به نماد مختلط نگاشت می‌شوند و نویز گاوسی سفید افزودنی به هر دو مؤلفه هم‌فاز (I) و تربیعی (Q) اضافه می‌گردد. BPSK هر بیت را به $1 \pm j$ می‌برد. 16-QAM هر چهار بیت را به یک نقطه از شبکه 4×4 با نگاشت Gray می‌نگارد؛ نگاشت Gray از این جهت اهمیت دارد که نقاط همسایه در صورت فلکی تنها در یک بیت اختلاف دارند، پس یک خطای تصمیم معمولاً تنها یک بیت را خراب می‌کند و نه چهار بیت را.

تعریف SNR و چرا E_b/N_0 انتخاب شد: در نسخه اولیه، نویز BPSK بر پایه انرژی هر بیت (E_b/N_0) و نویز QAM-16 بر پایه انرژی هر نماد (E_s/N_0) تنظیم می‌شد. چون هر نماد QAM-16 چهار بیت حمل می‌کند، این ناهمگونی یک اختلاف حدود 6.02 dB ایجاد می‌کرد و مقایسه دو مدولاسیون را بی‌معنا می‌ساخت. QAM-16: در نمودارها بی‌دلیل خوب به نظر می‌رسید. اکنون هر دو بر پایه انرژی هر بیت/اطلاعاتی تنظیم می‌شوند. با این تعریف مشترک، QAM-16 در E_b/N_0 برابر به‌طور قابل اندازه‌گیری نویزتری از BPSK است، و این دقیقاً بهایی است که در ازای چهار بیت بر نماد پرداخت می‌شود. برچسب‌های رابط گرافیکی هم به E_b/N_0 تغییر کردند تا عدد نمایش‌داده‌شده با کمیتی که واقعاً تنظیم می‌شود یکی باشد.

۴. لایه پیوند داده

۴.۱. فریم‌بندی، Bit Stuffing و مرزبندی در گیرنده

ساختار فریم عبارت است از پرچم آغاز، سرآیند (شماره ترتیب ۸ بیتی، نوع ۸ بیتی، CRC-16 سرآیند)، بار مفید، CRC-32 و پرچم پایان. جدا کردن CRC سرآیند از CRC بدنه یک انتخاب عامدانه است: اگر بار مفید خراب شود ولی سرآیند سالم بماند، گیرنده هنوز می‌داند این فریم کدام فریم بوده و می‌تواند یک NAK هدفمند بفرستد، در حالی که بدون آن تنها راه بازیابی، انتظار برای تایم‌اوت فرستنده بود.

مهم‌ترین تکمیل این بخش، پیاده‌سازی **مرزبندی فریم در گیرنده** است. پیش از این، گیرنده یک آرایه بیت را دریافت می‌کرد که از قبل دقیقاً یک فریم بود؛ در آن حالت Bit Stuffing هیچ نقش عملی نداشت و فقط یک تشریفات بود. اکنون گیرنده یک جریان بیت می‌گیرد و باید خودش مرزها را با جست‌وجوی الگوی پرچم پیدا کند. اینجاست که Bit Stuffing باربر می‌شود: چون

درج‌کننده تضمین می‌کند شش یک متوالی هرگز در بدنه ظاهر نشود، هر بار که الگوی 01111110 دیده می‌شود قطعاً یک مرز واقعی است و نه داده‌ای که تصادفاً شبیه پرچم شده است.

این پیاده‌سازی حالت‌دار است: فریمی که در دو نوبت خواندن برسد تا رسیدن پرچم پایانش در بافر می‌ماند، و بیت‌هایی که پیش از یک پرچم قرار می‌گیرند به‌عنوان هزینه همگام‌سازی مجدد شمرده می‌شوند نه آنکه بی‌صدا دور ریخته شوند. نتیجه در رابط گرافیکی قابل مشاهده است: در کانال تمیز شمارنده «Flag search» تعداد فریم‌های یافته را برابر تعداد ارسال‌ها نشان می‌دهد و تعداد همگام‌سازی مجدد صفر است؛ با بالا بردن نرخ خطا، همگام‌سازی مجدد شروع می‌شود.

اصلاح یک باگ ریشه‌ای در Bit Unstuffing: نسخه اولیه بیت بعد از پنج «یک» را تنها در صورتی حذف می‌کرد که آن بیت هنوز صفر باشد. اما اگر نویز همان بیت درج‌شده را به یک تبدیل می‌کرد، حذف انجام نمی‌شد، یک بیت اضافه در جریان می‌ماند و تمام بیت‌های بعدی یک خانه جابه‌جا می‌شدند؛ یعنی یک خطای تکبیتی کل هم‌ترازی فریم را نابود می‌کرد و Hamming و CRC هیچ شانس نداشتند. اکنون حذف بی‌قید و شرط انجام می‌شود: بیت درج‌شده هیچ اطلاعاتی حمل نمی‌کند، پس دور ریختن آن بدون توجه به مقدارش هم‌ترازی فریم را حفظ می‌کند و خطا موضعی می‌ماند تا لایه‌های تصحیح خطا بتوانند کارشان را انجام دهند.

درباره حفاظت‌نشدن پرچم: پرچم عامدانه بیرون از پوشش Hamming و CRC است و این یک نقص نیست بلکه اجتناب‌ناپذیر است — گیرنده باید پیش از هر رمزگشایی مرز فریم را بیابد، پس داده‌ای که مرز را مشخص می‌کند نمی‌تواند خودش نیازمند رمزگشایی باشد. راه‌حل درست، حفاظت از پرچم نیست بلکه تحمل خرابی آن است: فریمی که پرچمش آسیب ببیند یافته نمی‌شود، تلف‌شده به شمار می‌آید و تایمر فرستنده آن را بازیابی می‌کند. این همان رفتاری است که در HDLC واقعی هم دیده می‌شود و اکنون در شبیه‌ساز هم شمارش و در لاگ گزارش می‌شود.

۴.۲. تصحیح و تشخیص خطا

Hamming (12,8) روی هر بلوک هشت‌بیتی اعمال می‌شود و هر خطای تکبیتی را تصحیح می‌کند **CRC-32**. خطاهای انبوه و رشته‌ای را که از توان تصحیح Hamming فراتر می‌روند تشخیص می‌دهد. این ترتیب اهمیت دارد: نخست Unstuffing، سپس تصحیح Hamming و در پایان بررسی CRC. اگر CRC پیش از تصحیح بررسی شود، فریم‌هایی که Hamming می‌توانست نجات دهد بی‌جهت دور ریخته می‌شوند.

۴.۳. کنترل خطا و جریان Selective Repeat :

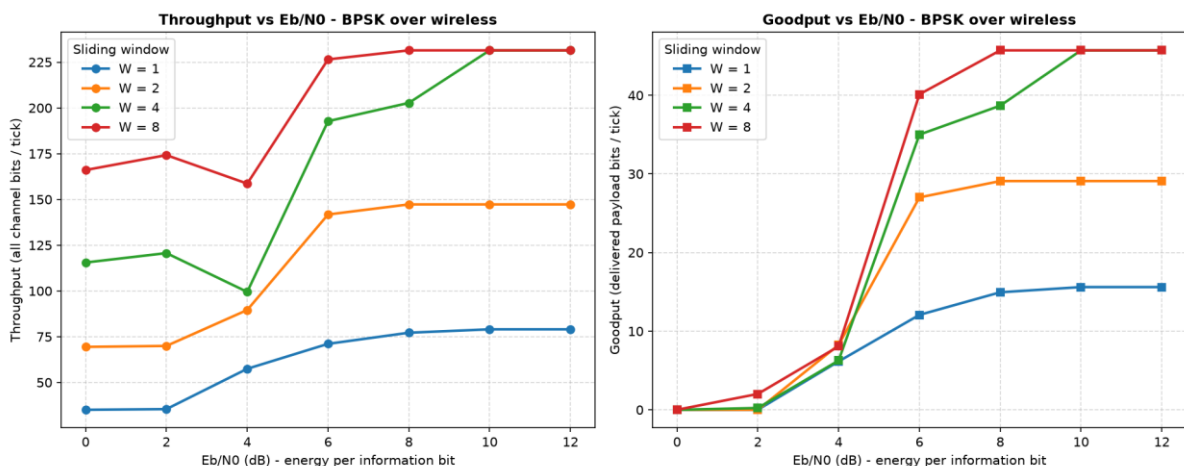
پنجره لغزان با تایمر مستقل برای هر فریم، بافر کردن فریم‌های خارج از ترتیب، تحویل مرتب به لایه بالا، تشخیص تکرار و بوجه محدود تلاش مجدد پیاده شده است. نکته‌ای که در پیاده‌سازی اهمیت داشت مدیریت فضای شماره ترتیب پیمانه‌ای است: شماره ترتیب هشت بیتی است و باید نسبت به مبنای پنجره تفسیر شود، وگرنه پس از سرریز شماره‌ها فریم‌های درست به‌اشتباه بیرون از پنجره تشخیص داده می‌شوند.

۵. لایه امنیت

رمزنگاری با XOR چرخشی و کلید پویا در بالاترین لایه و پیش از فریم‌بندی انجام می‌شود، تا همان‌طور که صورت مسئله می‌خواهد بدنه فریم‌های روی کانال کاملاً ناخوانا باشد. بابت کلید به اندازه اندیس موقعیت چرخش می‌کند، در نتیجه یک بایت یکسان در دو موقعیت مختلف به دو بایت رمز مختلف نگاشت می‌شود و تحلیل فراوانی ساده کارساز نیست. رمزگشایی در گیرنده تنها پس از موفقیت CRC و تصحیح Hamming و در بالاترین لایه انجام می‌گیرد.

۶. تحلیل نمودارهای Goodput و Throughput

دو کمیت اندازه‌گیری می‌شوند **Throughput**. همه بیت‌هایی است که روی کانال قرار می‌گیرند، شامل سرآیند، بیت‌های توازن، پرچم، بازارسال‌ها و فریم‌های کنترلی **Goodput**. فقط بار مفیدی است که سالم و مرتب به لایه بالا تحویل داده می‌شود. تفاوت این دو، تمام هزینه‌ای است که پروتکل برای قابل‌اعتماد بودن می‌پردازد.



جارتوب BPSK روی کانال بی‌سیم، پیام آزمون ۸۰ بایتی، قطعه ۸ بایتی، تأخیر انتشار ۲ تیک، میانگین سه اجرا برای هر نقطه.

W = 8	W = 4	W = 2	W = 1	Eb/N0 (dB)
166 / 0.0 (0%)	116 / 0.0 (0%)	69 / 0.0 (0%)	35 / 0.0 (0%)	0
174 / 2.0 (20%)	121 / 0.2 (3%)	70 / 0.0 (0%)	35 / 0.0 (0%)	2
159 / 8.1 (60%)	99 / 6.3 (67%)	90 / 8.3 (73%)	57 / 6.2 (100%)	4
226 / 40.1 (100%)	193 / 35.0 (100%)	142 / 27.0 (100%)	71 / 12.1 (100%)	6
231 / 45.7 (100%)	203 / 38.7 (100%)	147 / 29.1 (100%)	77 / 14.9 (100%)	8
231 / 45.7 (100%)	231 / 45.7 (100%)	147 / 29.1 (100%)	79 / 15.6 (100%)	10
231 / 45.7 (100%)	231 / 45.7 (100%)	147 / 29.1 (100%)	79 / 15.6 (100%)	12

هر خانه به‌ترتیب شامل **Goodput** / Throughput (درصد فریم‌های تحویل‌شده) بر حسب بیت بر تیک است. هر عدد میانگین سه اجرای مستقل است.

۶.۱ ناحیه آبخار: جایی که Goodput دروغ می‌گوید

مهم‌ترین مشاهده در سمت چپ نمودار است. در Eb/N0 برابر ۰ و ۲ دسی‌بل، Goodput برای همه اندازه‌های پنجره تقریباً صفر است، در حالی که Throughput مقادیر بزرگی مانند ۱۶۶ بیت بر تیک را نشان می‌دهد. یعنی کانال کاملاً مشغول است و هیچ کار مفیدی انجام نمی‌شود؛ تمام آن ظرفیت خرج بازار سال فریم‌هایی می‌شود که هرگز سالم نمی‌رسند. این تفاوت، دلیل وجودی سنجش هر دو کمیت است. اگر تنها Throughput گزارش می‌شد، یک لینک کاملاً از کار افتاده «پرمصرف و سالم» به نظر می‌رسید. همچنین توجه کنید که در این ناحیه Throughput با افزایش پنجره زیاد می‌شود (از ۳۵ برای W=1 به ۱۶۶ برای W=8) بی‌آنکه Goodput تغییری کند؛ پنجره بزرگ‌تر در کانال بد فقط شدت اتلاف را بالا می‌برد.

۶.۲. اثر اندازه پنجره و بازده نزولی آن

در ناحیه سالم، افزایش پنجره Goodput را بالا می‌برد اما نه به‌صورت خطی. در $E_b/N_0 = 6 \text{ dB}$ مقادیر Goodput به‌ترتیب 12.1، 27.0، 35.0 و 40.1 بیت بر تیک برای پنجره‌های ۱، ۲، ۴ و ۸ است. گام نخست (از ۱ به ۲) بیش از دو برابر بهبود می‌دهد، در حالی که گام آخر (از ۴ به ۸) تنها حدود 15% اضافه می‌کند.

دلیل این اشباع، حاصل‌ضرب پهنای‌باند در تأخیر است. با تأخیر انتشار دو تیک، پنجره فقط تا جایی سود می‌دهد که «زمان مرده» انتظار برای ACK را پر کند. وقتی پنجره به اندازه‌ای رسید که فرستنده پیش از رسیدن نخستین ACK بی‌کار نمی‌ماند، بزرگتر کردن آن چیزی اضافه نمی‌کند چون گلوگاه دیگر پنجره نیست. در بالای 10 dB هر دو پنجره ۴ و ۸ روی همان مقدار 45.7 بیت بر تیک اشباع می‌شوند و این تفسیر را تأیید می‌کنند.

یک مشاهده صادقانه دیگر: در 4 dB ترتیب یکنوا نیست — پنجره ۲ حدود 73% فریم‌ها را تحویل می‌دهد در حالی که پنجره ۴ فقط 67%. علت آن است که پنجره بزرگتر فریم‌های بیشتری را همزمان در یک کانال بد رها می‌کند و در نتیجه تعداد بیشتری از آن‌ها بودجه محدود تلاش مجدد را تمام می‌کنند و «تلف‌شده» اعلام می‌شوند. یعنی در آستانه آشبار، پنجره بزرگتر می‌تواند زیان‌بار باشد.

۶.۳. سقف بازدهی: چرا حدود ۲۰ درصد و نه بیشتر

نکته‌ای که در جدول برجسته است؛ نسبت Goodput به Throughput در تمام ناحیه سالم و برای همه اندازه‌های پنجره تقریباً ثابت و برابر 19.8% است. ثابت بودن این نسبت نشان می‌دهد که سقف بازدهی یک اثر کانالی یا پروتکلی نیست، بلکه ساختاری است. بودجه بیتی یک فریم موفق با قطعه ۸ بایتی چنین است:

توضیح	سهم	بیت	جزء
داده کاربر پس از رمزنگاری	20.0%	64	بار مفید (Payload)
شماره ترتیب، نوع، CRC-16 سرآیند و CRC-32 بدنه	20.0%	64	سرآیند و CRC-32 فریم داده
بسط 8/12 روی ۱۲۸ بیت، یعنی ۵۰% افزایش	20.0%	64	بیت‌های توازن Hamming فریم داده
دو پرچم ۸ بیتی 01111110	5.0%	16	پرچم‌های فریم داده
ACK هم یک فریم کامل است	20.0%	64	سرآیند و CRC فریم ACK
بسط 8/12 روی ۶۴ بیت	10.0%	32	بیت‌های توازن Hamming فریم ACK
ACK نیز مرزبندی می‌شود	5.0%	16	پرچم‌های فریم ACK
بازدهی نظری $= 320 \div 64 = 20.0\%$	100%	320	مجموع بیت‌های روی کانال برای یک فریم موفق

عدد نظری 20.0% با مقدار اندازه‌گیری‌شده 19.8% مطابقت دارد؛ اختلاف کوچک از بیت‌های Bit Stuffing است که به داده وابسته‌اند و در محاسبه بالا لحاظ نشده‌اند.

این تحلیل مسیر بهبود را هم مشخص می‌کند. بزرگترین سهم‌ها ثابت‌اند و به اندازه بار مفید بستگی ندارند: سرآیند، CRC و کل فریم ACK. بنابراین بزرگتر کردن قطعه داده این هزینه ثابت را سرشکن می‌کند. با قطعه ۳۲ بایتی، بار مفید 256 بیت می‌شود و مجموع بیت‌های کانال به 608 می‌رسد، یعنی بازدهی حدود 42% — بیش از دو برابر. در مقابل، بسط ۵۰ درصدی

Hamming بهایی است که آگاهانه پرداخت می‌شود. همان بسط است که اجازه می‌دهد خطاهای تکبیتی بدون هیچ بازارسالی تصحیح شوند، و در نزدیکی ناحیه آبشار همین ویژگی تفاوت میان لینک کارآمد و لینک از کار افتاده است.

۷. اعتبارسنجی

درستی پیاده‌سازی با 34 آزمون خودکار بررسی می‌شود که همه موفق هستند. این آزمون‌ها فقط مسیر خوش‌بینانه را پوشش نمی‌دهند؛ بخش عمده آن‌ها دقیقاً همان باگ‌هایی را هدف می‌گیرند که در جریان این کار پیدا و اصلاح شدند، تا بازگشت آن‌ها ممکن نباشد. نمونه‌هایی از آنچه آزموده می‌شود:

- رفت‌وبرگشت کامل رمزنگاری و ماهیت خودمعکوس آن، و وابستگی رمز به موقعیت بایت.
- رفت‌وبرگشت B8ZS روی جریان‌های تصادفی، و به‌طور خاص جانشینی‌ای که در ابتدای جریان رخ می‌دهد (حالتی که دیگر اولیه در آن شکست می‌خورد).
- کران رشته بدون پالس برای هر دو کد خطی.
- همارز بودن واحد Eb/N_0 میان دو مدولاسیون، و نویزی‌تر بودن QAM-16 در Eb/N_0 برابر.
- رد شدن ترکیب‌های نامعتبر کانال و مود با خطای صریح.
- یافتن مرز فریم در جریان پیوسته، فریم تقسیم‌شده میان دو خواندن، و بازیابی فریم بعدی پس از آسیب دیدن یک پرچم.
- حفظ هم‌ترازی فریم هنگام خراب شدن بیت درج‌شده در Bit Stuffing.
- امکان ایجاد و حذف پالس در مدل خطای کانال سیمی (اینکه صفرها مصون نیستند).
- تصحیح خطای تکبیتی با Hamming و تشخیص خطای انبوه با CRC-32.
- بازیابی ACK گم‌شده با تایمر، و پایان‌پذیری اجرا در برابر بودجه تلاش مجدد.

۸. نتیجه‌گیری

شبیه‌ساز یک مسیر کامل و کارکردی از متن ورودی تا متن بازیابی‌شده را پوشش می‌دهد و هر لایه در شرایطی که برای آن طراحی شده واقعاً وارد عمل می‌شود. سه دستاورد اصلی این مرحله از کار عبارت‌اند از: یکسان‌سازی تعریف نویز روی Eb/N_0 که مقایسه مدولاسیون‌ها را معنادار کرد؛ پیاده‌سازی مرزبندی فریم در گیرنده که Bit Stuffing را از یک تشریفات به سازوکاری باربر تبدیل کرد؛ و واقعی کردن مدل خطای کانال سیمی که پیش‌تر بیشتر نمادهایش از نویز مصون بودند.

تحلیل کارایی نشان داد که پرسش «این لینک چقدر سریع است» بدون تفکیک Throughput از Goodput پاسخ گمراه‌کننده می‌دهد، که اندازه پنجره تنها تا حاصل‌ضرب پهنای‌باند در تأخیر سود می‌دهد و پس از آن در کانال بد حتی زیان‌بار می‌شود، و اینکه سقف بازدهی حدود ۲۰ درصد یک محدودیت ساختاری ناشی از سرآیند ثابت، ترافیک ACK و بسط Hamming است که با بزرگ‌تر کردن قطعه داده قابل بهبود است.